

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

Лабораторная работа 7

По дисциплине Объектно-ориентированное программирование

Тема работы Создание иерархии классов

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и
системы связи

Образовательная программа Программирование в инфокоммуникацион-
ных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Иванов С.Е.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 УПРАЖНЕНИЕ 1.....	4
2 УПРАЖНЕНИЕ 2.....	6
3 УПРАЖНЕНИЕ 3.....	9
4 УПРАЖНЕНИЕ 4.....	10
5 УПРАЖНЕНИЕ 5.....	12
6 УПРАЖНЕНИЕ 6.....	15
7 УПРАЖНЕНИЕ 7.....	18
7.1 Код-ревью.....	20
ЗАКЛЮЧЕНИЕ	23

ВВЕДЕНИЕ

Отчёт по выполнению лабораторной работы. В тексте работы представлены результаты выполнения семи упражнений.

Первые 4 упражнения повествуют о наследовании, второе - про использование конструкторов и ссылки на базовый родительский класс, третье - про переопределение методов и `virtual`, четвёртое - про абстрактные методы.

В 5 упражнении на примере точек и отрезка реализуется отношение включения. В 6 упражнении - отношение ассоциации. Упражнение 7 для самостоятельной работы.

Цель - изучение наследования как важного элемента объектно-ориентированного программирования и приобретение навыков реализации иерархии классов.

1 УПРАЖНЕНИЕ 1

Сначала был написан базовый класс «Item» (рисунок 1).

```
1  using System;
2
3  namespace MyClass
4  {
5      3 references
6      internal class Item
7      {
8          protected long invNumber;
9          protected bool taken;
10
11          0 references
12          public long GetInvNumber()
13          {
14              return invNumber;
15          }
16
17          1 reference
18          public bool IsAvailable()
19          {
20              return !taken;
21          }
22
23          1 reference
24          public void Take()
25          {
26              taken = true;
27          }
28
29          0 references
30          public void Return()
31          {
32              taken = false;
33          }
34
35          1 reference
36          public void Show()
37          {
38              Console.WriteLine(
39                  "Состояние единиц хранения\n Инвентарный номер: {0}\n Наличие: {1}",
40                  invNumber, taken
41              );
42          }
43      }
44  }
```

Рисунок 1 — Класс «Item»

Классу «Book» был задан родительский класс и дополнил методами (рисунок 2).

```

0 references
new public void Show()
{
    Console.WriteLine(
        "\nКнига\n Автор: {0}\n Название: {1}\n Издатель: {2}\n Год издания: {3}\n {4} стр.\n Стоимость аренды: {5}\n",
        this.author, this.title, this.publisher, this.year, this.pages, Book.price
    );
}

0 references
public bool TakeItem()
{
    if (IsAvailable())
    {
        Take();
        return true;
    }
    return false;
}

```

Рисунок 2 — Изменённый и добавленный методы в класс «Book»

Результат работы при выводе «Show» класса «Item» представлен на рисунке 3.

```

Microsoft Visual Studio Debug Console

Состояние единицы хранения
Инвентарный номер: 0
Наличие: False

D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\MyClass\bin\Debug\MyClass.exe (process 33160)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 3 — Консольный вывод программы

2 УПРАЖНЕНИЕ 2

У класс «Item» был добавлен конструктор (рисунок 4).

```
0 references
public Item(long invNumber, bool taken)
{
    this.invNumber = invNumber;
    this.taken = taken;
}

1 reference
public Item()
{
    taken = false;
}
```

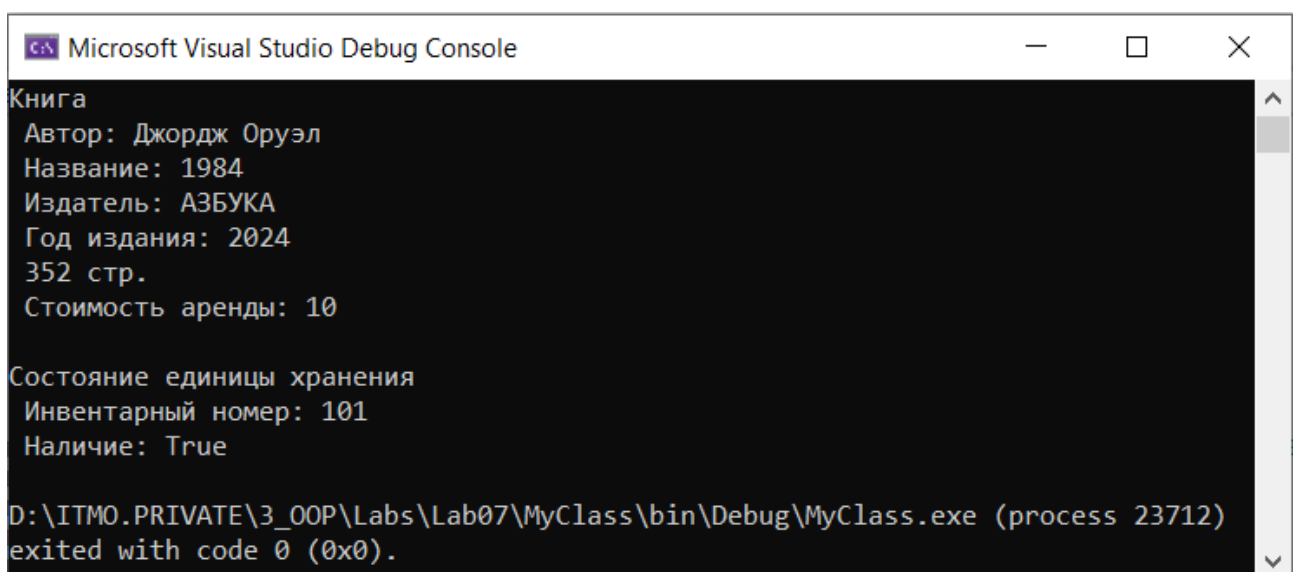
Рисунок 4 — Новый конструктор в классе «Item»

В классе «Book» был дополнен конструктор и метод «Show» (рисунок 2).

```
1 reference
new public void Show()
{
    Console.WriteLine(
        "\nКнига\n Автор: {0}\n Название: {1}\n Издатель: {2}\n Год издания: {3}\n {4} стр.\n Стоимость аренды: {5}\n",
        author, title, publisher, year, pages, price
    );
    base.Show();
}
```

Рисунок 5 — Изменённый метод в классе «Book»

Результат работы при выводе «Show» класса «Book» представлен на рисунке 6.



```
Microsoft Visual Studio Debug Console

Книга
Автор: Джордж Оруэл
Название: 1984
Издатель: АЗБУКА
Год издания: 2024
352 стр.
Стоимость аренды: 10

Состояние единицы хранения
Инвентарный номер: 101
Наличие: True

D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\MyClass\bin\Debug\MyClass.exe (process 23712)
exited with code 0 (0x0).
```

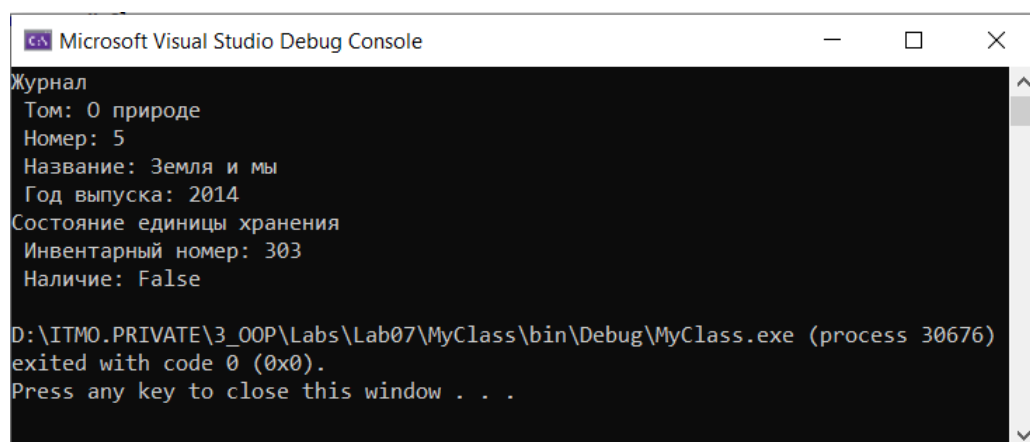
Рисунок 6 — Консольный вывод программы

Далее был добавлен ещё один производный класс журнала (рисунок 7).

```
1  using System;
2
3  namespace MyClass
4  {
5      4 references
6      internal class Magazine : Item
7      {
8          private string volume;
9          private int number;
10         private string title;
11         private int year;
12
13         1 reference
14         public Magazine(string volume, int number, string title,
15             int year, long invNumber, bool taken) : base(invNumber, taken)
16         {
17             this.volume = volume;
18             this.number = number;
19             this.title = title;
20             this.year = year;
21         }
22
23         0 references
24         public Magazine() {}
25
26         1 reference
27         new public void Show()
28         {
29             Console.WriteLine(
30                 "\nЖурнал\n Том: {0}\n Номер: {1}\n Название: {2}\n Год выпуска: {3}",
31                 volume, number, title, year);
32             base.Show();
33         }
34     }
35 }
```

Рисунок 7 — Класс «Magazine»

Результат работы при выводе «Show» класса «Magazine» представлен на рисунке 8.



```
Microsoft Visual Studio Debug Console

Журнал
Том: 0 природе
Номер: 5
Название: Земля и мы
Год выпуска: 2014
Состояние единицы хранения
Инвентарный номер: 303
Наличие: False

D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\MyClass\bin\Debug\MyClass.exe (process 30676)
exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 8 — Консольный вывод программы

Для проекта средствами IDE была сгенерирована диаграмма классов (рисунок 9).

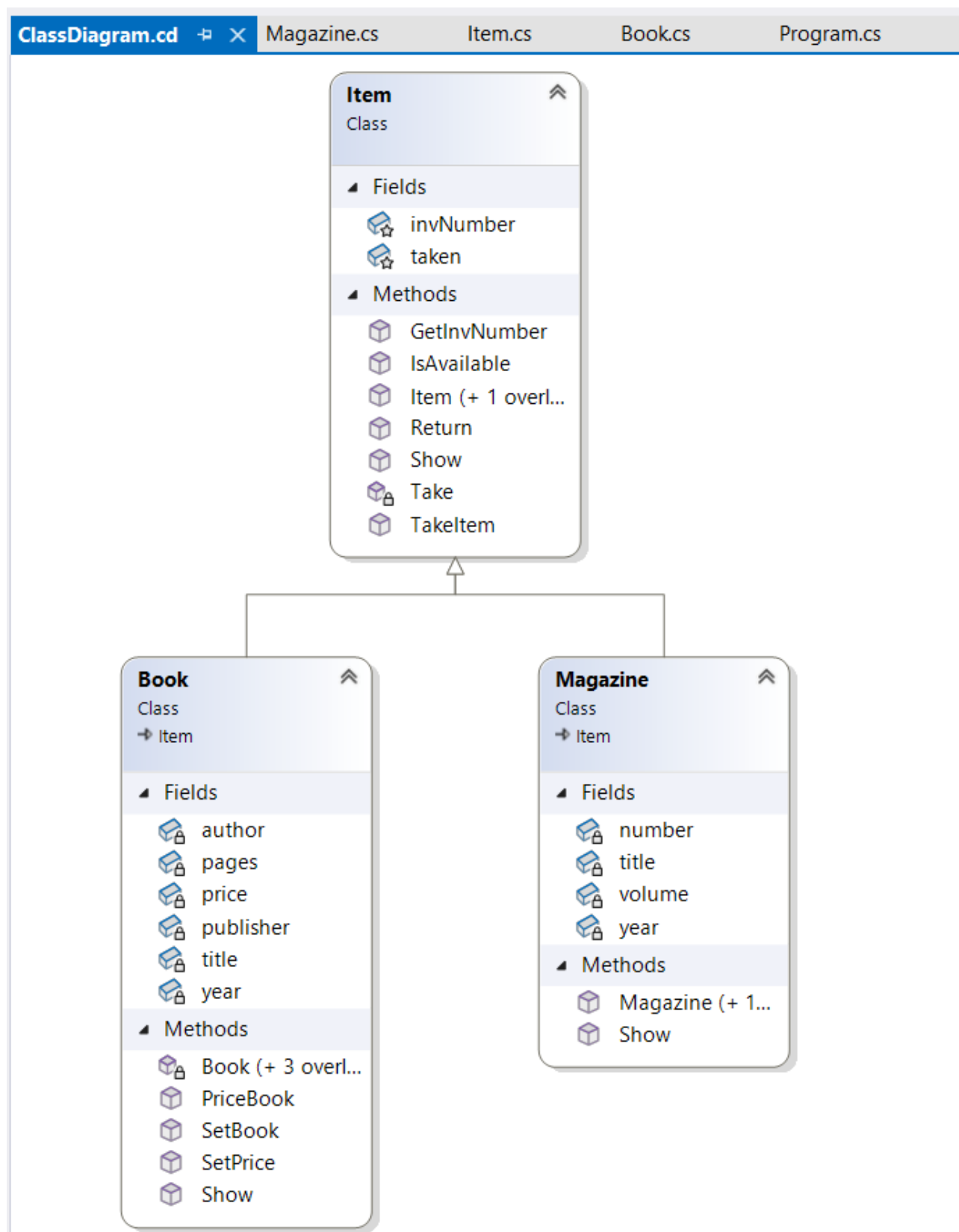


Рисунок 9 — Диаграмма классов

3 УПРАЖНЕНИЕ 3

В 3 упражнении были переопределены несколько методов (рисунок 10).

```
// Упражнения 3 - 4
Console.WriteLine("\n===== УПРАЖНЕНИЯ 3 - 4 =====\n");
Console.WriteLine("Тестирование полиморфизма");
Item it;

it = b2;
it.TakeItem();
it.Return();
it.Show();

it = mag1;
it.TakeItem();
it.Return();
it.Show();
```

Рисунок 10 — Тестирование переопределённых методов

4 УПРАЖНЕНИЕ 4

Базовый класс «Item» был переписан как абстрактный (рисунок 11).

```
1      using System;
2
3      namespace MyClass
4      {
5          7 references
6          internal abstract class Item
7          {
8              protected long invNumber;
9              protected bool taken;
10
11              2 references
12              public Item(long invNumber, bool taken)
13              {
14                  this.invNumber = invNumber;
15                  this.taken = taken;
16              }
17
18              0 references
19              public Item()
20              {
21                  taken = false;
22              }
23
24              0 references
25              public long GetInvNumber()
26              {
27                  return invNumber;
28              }
29
30              1 reference
31              public bool IsAvailable()
32              {
33                  return !taken;
34              }
35
36              1 reference
37              private void Take()
38              {
39                  taken = true;
40              }
41
42              3 references
43              public bool TakeItem()
44              {
45                  if (IsAvailable())
46                  {
47                      Take();
48                      return true;
49                  }
50                  return false;
51              }
52
53              4 references
54              abstract public void Return();
55
56              8 references
57              public virtual void Show()
58              {
59                  Console.WriteLine(
60                      "Состояние единицы хранения\n Инвентарный номер: {0}\n Наличие: {1}",
61                      invNumber, !taken
62                  );
63              }
64          }
65      }
```

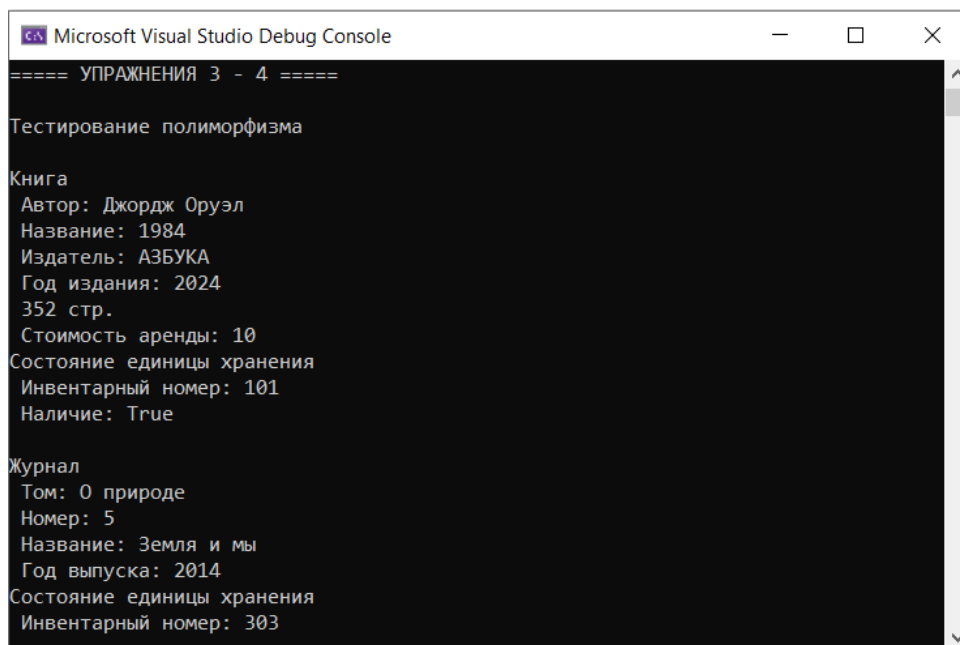
Рисунок 11 — Новый класс «Item»

Код запуска представлен на рисунке 12.

```
1  using System;
2  namespace MyClass
3  {
4      0 references
5      internal class Program
6      {
7          0 references
8          static void Main(string[] args)
9          {
10             // Lab06
11             // Book b1 = new Book("Стивен Хокинг", "Краткая история времени", "АСТ МОСКВА", 320, 2022);
12             // Book b2 = new Book("Джордж Оруэл", "1984", "АЗБУКА", 352, 2024);
13
14             // Упражнение 1
15             //Console.WriteLine("==== УПРАЖНЕНИЕ 1 ==== \n");
16             //Item item1 = new Item();
17             //item1.Show();
18
19             // Упражнение 2
20             Console.WriteLine("\n==== УПРАЖНЕНИЕ 2 =====");
21             Book b2 = new Book("Джордж Оруэл", "1984", "АЗБУКА", 352, 2024, 101, false);
22             b2.TakeItem();
23             b2.Show();
24             Magazine mag1 = new Magazine("0 природе", 5, "Земля и мы", 2014, 303, false);
25             mag1.Show();
26
27             // Упражнения 3 - 4
28             Console.WriteLine("\n==== УПРАЖНЕНИЯ 3 - 4 ===== \n");
29             Console.WriteLine("Тестирование полиморфизма");
30             Item it;
31
32             it = b2;
33             it.TakeItem();
34             it.Return();
35             it.Show();
36
37             it = mag1;
38             it.TakeItem();
39             it.Return();
40             it.Show();
41         }
42     }
43 }
```

Рисунок 12 — Тестирование упражнений 1-4

Результат работы представлен на рисунке 13.



```
Microsoft Visual Studio Debug Console
==== УПРАЖНЕНИЯ 3 - 4 ====
Тестирование полиморфизма
Книга
Автор: Джордж Оруэл
Название: 1984
Издатель: АЗБУКА
Год издания: 2024
352 стр.
Стоимость аренды: 10
Состояние единицы хранения
Инвентарный номер: 101
Наличие: True
Журнал
Том: 0 природе
Номер: 5
Название: Земля и мы
Год выпуска: 2014
Состояние единицы хранения
Инвентарный номер: 303
```

Рисунок 13 — Консольный вывод программы

5 УПРАЖНЕНИЕ 5

Был создан проект «MyClassLine», в нём добавлен класс «Point», состоящий из двух координат и метода расчёта расстояния до другого объекта «Point» (рисунок 14).

```
1  using System;
2
3  namespace MyClassLine
4  {
5      11 references
6      class Point
7      {
8          private double x;
9          private double y;
10
11          0 references
12          public Point() { }
13          2 references
14          public Point(double x, double y)
15          {
16              this.x = x;
17              this.y = y;
18          }
19          2 references
20          public void Show()
21          {
22              Console.WriteLine("Точка с координатами: ({0}, {1})", x, y);
23          }
24          1 reference
25          public double Length(Point p)
26          {
27              return Math.Sqrt(Math.Pow(this.x - p.x, 2) + Math.Pow(this.y - p.y, 2));
28          }
29          0 references
30          public override string ToString()
31          {
32              return x + ", " + y;
33          }
34      }
35  }
```

Рисунок 14 — Класс «Point»

Затем был добавлен класс «Line» реализующий отрезок, концы которого являются точками «Point» (рисунок 15). Вместе они образуют отношение агрегации.

```

1  using System;
2
3  namespace MyClassLine
4  {
5      2 references
6      class Line
7      {
8          private Point pStart;
9          private Point pEnd;
10
11          0 references
12          public Line() { }
13          1 reference
14          public Line(Point pStart, Point pEnd)
15          {
16              this.pStart = pStart;
17              this.pEnd = pEnd;
18          }
19          0 references
20          public void Show()
21          {
22              Console.WriteLine("Отрезок с координатами: ({0}) - ({1})", pStart, pEnd);
23          }
24          1 reference
25          public double Length()
26          {
27              return pStart.Length(pEnd);
28          }
29      }
30  }

```

Рисунок 15 — Класс «Line»

Код запуска представлен на рисунке 16.

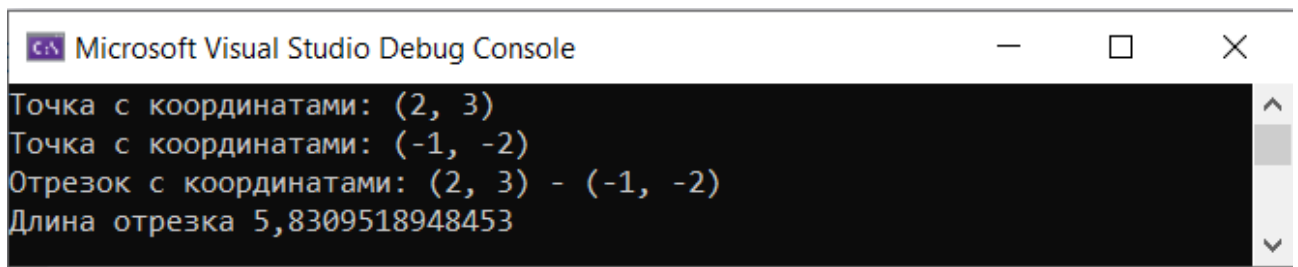
```

1  using System;
2
3  namespace MyClassLine
4  {
5      0 references
6      internal class Program
7      {
8          0 references
9          static void Main(string[] args)
10         {
11             Point p1 = new Point(2, 3);
12             Point p2 = new Point(-1, -2);
13             p1.Show();
14             p2.Show();
15
16             Line line = new Line(p1, p2);
17             line.Show();
18
19             Console.WriteLine("Длина отрезка {0}", line.Length());
20         }
21     }
22 }

```

Рисунок 16 — Тестирующий код

Результат работы представлен на рисунке 17.

The image shows a screenshot of the Microsoft Visual Studio Debug Console window. The window has a title bar with the text "Microsoft Visual Studio Debug Console" and standard window control buttons (minimize, maximize, close). The console area has a black background with white text. The output consists of four lines: "Точка с координатами: (2, 3)", "Точка с координатами: (-1, -2)", "Отрезок с координатами: (2, 3) - (-1, -2)", and "Длина отрезка 5,8309518948453". A vertical scrollbar is visible on the right side of the console area.

```
Microsoft Visual Studio Debug Console
Точка с координатами: (2, 3)
Точка с координатами: (-1, -2)
Отрезок с координатами: (2, 3) - (-1, -2)
Длина отрезка 5,8309518948453
```

Рисунок 17 — Консольный вывод программы

6 УПРАЖНЕНИЕ 6

Оригинальные названия переменных из учебного пособия были изменены с транслитерации на английские слова. Был создан проект «Game», в нём добавлен класс «GameDice» - игральная кость, состоящий из конструктор и метода «random» (рисунок 18).

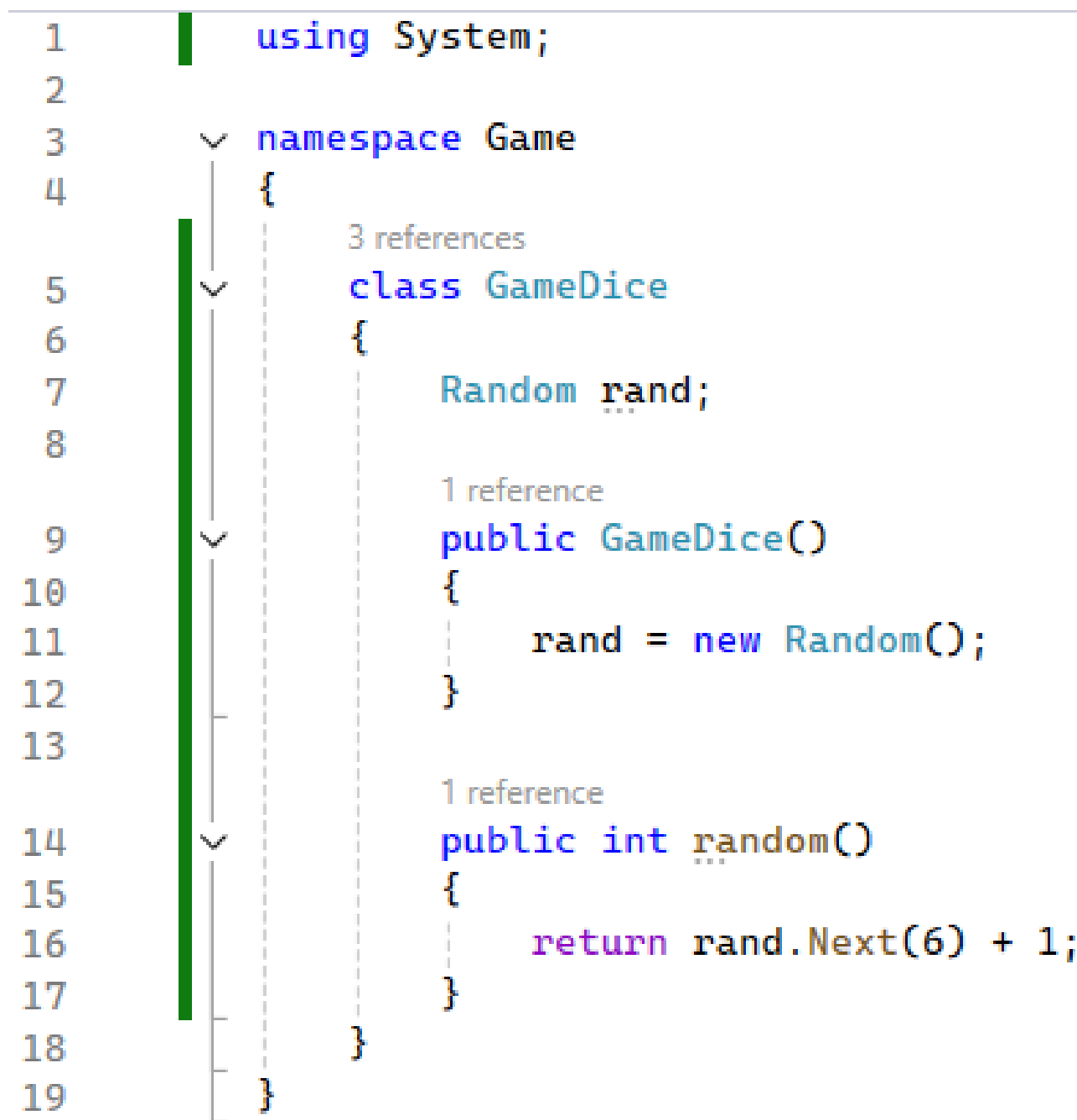


Рисунок 18 — Класс «GameDice»

Далее был добавлен класс «Gamer», реализующий игрока, использующего функционал сеанса - игрового кубика (рисунок 19).

```

1  namespace Game
2  {
3      3 references
4      class Gamer
5      {
6          string Name;
7          GameDice Session;
8
9          1 reference
10         public Gamer(string name)
11         {
12             Name = name;
13             Session = new GameDice();
14
15             1 reference
16             public int GameSession()
17             {
18                 return Session.random();
19
20             1 reference
21             public override string ToString()
22             {
23                 return Name;
24             }
25         }
26     }
27 }

```

Рисунок 19 — Класс «Gamer»

Код запуска шести игр представлен на рисунке 20.

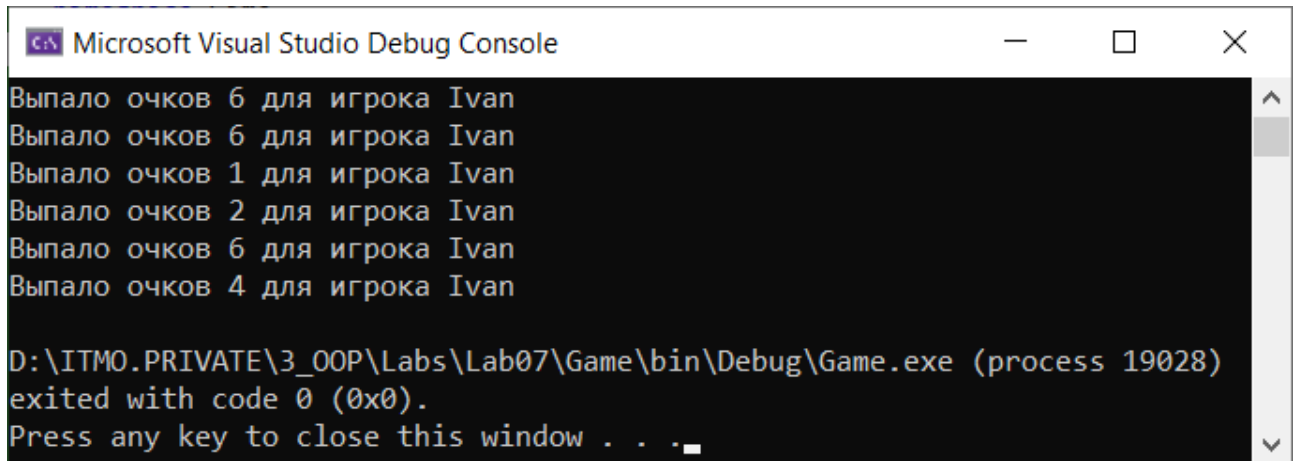

```

1  using System;
2
3  namespace Game
4  {
5      0 references
6      internal class Program
7      {
8          0 references
9          static void Main(string[] args)
10         {
11             Gamer g1 = new Gamer("Ivan");
12
13             for (int i = 1; i <= 6; i++)
14                 Console.WriteLine("Выпало очков {0} для игрока {1}", g1.GameSession(), g1.ToString());
15         }
16     }

```

Рисунок 20 — Тестирующий код

Результат работы представлен на рисунке 21.



```

Microsoft Visual Studio Debug Console
Выпало очков 6 для игрока Ivan
Выпало очков 6 для игрока Ivan
Выпало очков 1 для игрока Ivan
Выпало очков 2 для игрока Ivan
Выпало очков 6 для игрока Ivan
Выпало очков 4 для игрока Ivan
D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\Game\bin\Debug\Game.exe (process 19028)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 21 — Консольный вывод программы

7 УПРАЖНЕНИЕ 7

Был создан абстрактный класс прогрессии, в котором прописаны абстрактный метод получения элемента и первый элемент (рисунок 22).

```
1  namespace Progressions
2  {
3      3 references
4      abstract class Progression
5      {
6          protected double first;
7          3 references
8          public abstract double GetElement(int k);
9      }
10 }
```

Рисунок 22 — Абстрактный класс прогрессии

От абстрактного были унаследованы два вида прогрессий, различающиеся подсчётом элемента и полем разность / множитель (рисунки 23 и 24).

```
1  using System;
2
3  namespace Progressions
4  {
5      2 references
6      class ArithmeticProgression: Progression
7      {
8          private double difference;
9
10         1 reference
11         public ArithmeticProgression(double first, double difference)
12         {
13             this.first = first;
14             this.difference = difference;
15         }
16
17         2 references
18         public override double GetElement(int k)
19         {
20             if (k <= 0) throw new ArgumentOutOfRangeException();
21             return first + difference * (k - 1);
22         }
23     }
24 }
```

Рисунок 23 — Арифметическая прогрессия

```

1  using System;
2  namespace Progressions
3  {
4      2 references
5      class GeometricProgression: Progression
6      {
7          private double denominator;
8
9          1 reference
10         public GeometricProgression(double first, double denominator)
11         {
12             this.first = first;
13             this.denominator = denominator;
14
15         2 references
16         public override double GetElement(int k)
17         {
18             if (k <= 0) throw new ArgumentOutOfRangeException();
19             return first * Math.Pow(denominator, k - 1);
20         }
21     }
22 }

```

Рисунок 24 — Геометрическая прогрессия

Результат работы представлен на рисунках 25 и 26.

```

Microsoft Visual Studio Debug Console
Выберите вид прогрессии ['+' - арифмет., иначе - геометр.]: +
Введите первый элемент: 1
Введите разность: 2
Введите номер элемента: 8
Элемент номер 8 равен 15

D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\Progressions\bin\Debug\Progressions.exe
(process 4524) exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 25 — Арифметическая прогрессия

```

Microsoft Visual Studio Debug Console
Выберите вид прогрессии ['+' - арифмет., иначе - геометр.]:
Введите первый элемент: 3
Введите множитель: 2
Введите номер элемента: 9
Элемент номер 9 равен 768

D:\ITMO.PRIVATE\3_OOP\Labs\Lab07\Progressions\bin\Debug\Progressions.exe
(process 27456) exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 26 — Геометрическая прогрессия

Код для тестирования представлен на рисунке 27.

```
1  using System;
2
3  namespace Progressions
4  {
5      0 references
6      internal class Program
7      {
8          0 references
9          static void Main(string[] args)
10         {
11             try
12             {
13                 Console.WriteLine("Выберите вид прогрессии ['+' - арифмет., иначе - геометр.]: ");
14                 string type = Console.ReadLine();
15                 bool arifmet = type == "+";
16                 Console.WriteLine("Введите первый элемент: ");
17                 double first = double.Parse(Console.ReadLine());
18                 Console.WriteLine("Введите {0}: ", arifmet ? "разность" : "множитель");
19                 double d = double.Parse(Console.ReadLine());
20                 Console.WriteLine("Введите номер элемента: ");
21                 int k = int.Parse(Console.ReadLine());
22
23                 Progression progression;
24                 if (arifmet)
25                     progression = new ArithmeticProgression(first, d);
26                 else
27                     progression = new GeometricProgression(first, d);
28                 Console.WriteLine("Элемент номер {0} равен {1}", k, progression.GetElement(k));
29             }
30             catch (FormatException)
31             {
32                 Console.WriteLine("Некорректный ввод.");
33             }
34             catch (OverflowException e)
35             {
36                 Console.WriteLine("Слишком маленькое/большое число. {0}", e.Message);
37             }
38             catch (ArgumentOutOfRangeException e)
39             {
40                 Console.WriteLine("Номер элемента должен быть положительным числом!");
41             }
42         }
43     }
44 }
```

Рисунок 27 — Код для тестирования

7.1 Код-ревью

Резюме

Код реализует абстрактный класс `Progression` и два его наследника: `ArithmeticProgression` и `GeometricProgression`. Каждая из этих реализаций предоставляет метод для вычисления элемента прогрессии по заданному индексу. Основная программа позволяет пользователю выбрать тип прогрессии и вводить необходимые параметры.

Ошибка

Код корректно обрабатывает исключения, такие как `FormatException`, `OverflowException` и `ArgumentOutOfRangeException`. Однако, стоит рассмотреть возможность добавления более детализированных сообщений об ошибках, чтобы улучшить пользовательский опыт.

Стиль кода

Стиль кода в целом соответствует стандартам C#. Имена классов и методов написаны в `PascalCase`, что является хорошей практикой. Однако, можно улучшить читаемость, добавив комментарии к методам и классам, чтобы объяснить их назначение.

Структура кода

Структура кода логична и хорошо организована. Абстрактный класс `Progression` служит основой для конкретных реализаций, что соответствует принципам ООП. Однако, можно рассмотреть возможность вынесения логики ввода в отдельный класс или метод для улучшения модульности.

Читаемость

Читаемость кода хорошая, но можно улучшить форматирование, добавив пробелы между логическими блоками кода. Также стоит использовать более описательные имена переменных, чтобы сделать код более самодокументируемым.

Производительность

Код работает эффективно для небольших значений k . Однако, для больших значений геометрической прогрессии может возникнуть проблема с переполнением. Рекомендуется добавить проверку на переполнение при вычислении значений.

Масштабируемость

Код легко масштабируется, так как добавление новых типов прогрессий требует лишь создания нового класса, наследующего от `Progression`. Однако, стоит учитывать, что при добавлении новых типов может потребоваться изменение логики ввода в основной программе.

Безопасность

Код не содержит явных уязвимостей, но стоит учитывать возможность некорректного ввода данных пользователем. Рекомендуется использовать более строгую валидацию входных данных.

Обработка ошибок

Обработка ошибок реализована корректно, но можно улучшить информативность сообщений об ошибках. Например, можно указать, какой именно ввод был некорректным.

Заключение

В целом, код представляет собой хорошую реализацию классов для работы с арифметическими и геометрическими прогрессиями. Существуют некоторые области для улучшения, такие как обработка ошибок, читаемость и модульность. Рекомендуется внести предложенные изменения для повышения качества кода и улучшения пользовательского опыта.

ЗАКЛЮЧЕНИЕ

В ходе работы было изучены различные отношения между классами и рассмотрены конкретные примеры их реализации. Также познакомился с тернарным оператором.